Reproducing and Analyzing ROCKET for Time Series Classification

Kalyan Kumar Bhumaiahgari
Student ID: 25234940
COMP41850: AI for Time Series
School of Computer Science, University College Dublin
kalyan.bhumaiahgari@ucdconnect.ie

***Abstract*—This report presents a structured reproduction and analysis of ROCKET (RandOm Convolutional KErnel Transform), a state-of-the-art method for time series classification proposed by Dempster et al. [1]. Using the aeon Python library, the canonical ROCKET configuration was evaluated across 21 UCR benchmark datasets under 10 random seeds per experiment, with results compared against the paper's published figures. Two additional large-scale datasets were evaluated for scalability verification. The baseline reproduction achieved a mean accuracy of 0.8816 across 21 datasets, with a mean deviation of only −0.0016 relative to the paper's reported values. Five experimental conditions were investigated beyond the baseline: a classifier head ablation confirming Ridge regression as the optimal choice for ROCKET's high-dimensional feature space; an efficiency evaluation of MiniROCKET (mean accuracy 0.8924); a feature ablation reproducing the paper's PPV-versus-max finding (PPV wins on 12 of 21 datasets); and two proposed improvements consisting of two novel pooling operators, the Peak-Prevalence Product (PPP) and Proportion Above Mean (PAM), plus an adaptive kernel count heuristic. The adaptive kernel approach matched baseline accuracy within 0.0011 on average while reducing kernel counts by up to 90% on small datasets. Several practical engineering challenges were encountered and resolved, including tree-based classifier intractability in high-dimensional spaces and a data leakage bug in the PAM implementation.**

***Index Terms*—time series classification, ROCKET, UCR archive, reproducibility, novel pooling operators, adaptive kernels, aeon**

I. Motivation and Impact

A. The Problem with Existing Methods

Time series classification (TSC) is one of the most practically important problems in applied machine learning, with applications spanning healthcare (ECG arrhythmia detection), industrial monitoring (sensor anomaly detection), agriculture (crop classification from spectroscopy), and finance (regime detection in market data). Despite this practical relevance, achieving high classification accuracy with tractable training time remained genuinely difficult for many years.

For a long time, the field was dominated by one-nearest-neighbour classification with Dynamic Time Warping (1-NN DTW). While effective, pairwise DTW distance computation has quadratic complexity in both the number of examples and the series length, making it intractable at scale. Ensemble methods such as HIVE-COTE [2] addressed accuracy limitations but introduced even higher computational cost, with training complexity bounded by the Shapelet Transform component at $O(n^2 \cdot l^4)$. On the HandOutlines dataset (series length 2,709), this exceeds four days of computation on a single CPU [1].

Deep learning approaches including ResNet [3] and InceptionTime [4] brought training times down considerably through GPU acceleration, but require substantial labelled data and exhibit high variance across individual runs.

B. ROCKET as a Paradigm Shift

ROCKET [1] challenges the assumption that discriminative features must be either hand-crafted or learned via gradient descent. It demonstrates that a large number of random convolutional kernels, with randomised lengths drawn from $\{7, 9, 11\}$, normally distributed and mean-centred weights, uniformly sampled biases, exponentially sampled dilations, and random padding, when combined with a Ridge regression classifier, achieve state-of-the-art accuracy across the entire UCR archive.

The key insight is that 10,000 random kernels in aggregate provide extraordinary feature diversity. Each kernel produces two aggregate values: the maximum activation (analogous to global max pooling) and the Proportion of Positive Values (PPV), a novel pooling operation that measures what fraction of a time series activates a given kernel above a threshold. This produces a 20,000-dimensional feature vector per time series, classified using Ridge regression with generalised cross-validation. Training complexity is $O(k \cdot n \cdot l)$, linear in the number of kernels k, training examples n, and series length l.

C. Community Impact

The impact of ROCKET on the broader machine learning community has been substantial. Its publication catalysed an entire research direction in scalable TSC, with MiniROCKET [5], MultiROCKET, and Hydra [6] all building directly on its foundation. ROCKET has been integrated as a reference implementation into the aeon open-source library [7], which is the primary toolkit used by practitioners and researchers in this field.

II. Approach to Reproduction

A. Software and Hardware Environment

All experiments were implemented in Python 3.12 using the aeon library (version 1.4.0), scikit-learn

(version 1.6.1), and xgboost. Experiments were run on Google Colab using a standard CPU runtime. All code, raw results in CSV format, and generated figures are available at: https://github.com/kalyanbhumaiahgari/COMP41850-ROCKET-project-Reproduction

B. Dataset Selection and Justification

The original paper evaluated performance across 85 UCR archive datasets. To improve on a prior attempt limited to 7 datasets, this study was expanded to 21 small-to-medium UCR benchmark datasets spanning multiple domains, series lengths (24 to 1,882 timesteps), and training set sizes (20 to 500 examples). Dataset selection was guided by three principles: direct comparability with the paper's Appendix D Table 1, domain diversity (ECG, spectroscopy, motion, geology, household appliances), and coverage of both development and holdout datasets from the original paper's experimental protocol.

Two additional large-scale datasets were reserved for scalability verification only: HandOutlines (series length 2,709, the longest in the bake-off) and ElectricDevices (8,926 training examples, the largest training set). These were evaluated under Method A with 3 seeds each, as running 10 methods across 10 seeds on these datasets would have been computationally intractable on a Colab CPU runtime.

C. Experimental Protocol

The original paper reports mean results over 10 independent runs, each using a freshly sampled set of 10,000 random kernels. This study matches that protocol exactly, running all experiments across 10 distinct random seeds (seeds 0 through 9). The number of kernels was fixed at $k = 10{,}000$ for all methods except the adaptive kernel experiment (Method J). All datasets were loaded using standard UCR train/test splits with no additional preprocessing.

Ten experimental conditions were evaluated. Method A reproduced the canonical ROCKET configuration (Ridge classifier with generalised cross-validation). Methods B and C replaced the Ridge classifier with Random Forest and XGBoost respectively. Method D evaluated MiniROCKET. Methods E and F isolated the PPV and max features. Methods G, H, and I tested novel pooling operators. Method J evaluated an adaptive kernel count heuristic.

Due to the compute-intensive nature of running 10 methods across 21 datasets over 10 seeds, an automated state-caching pipeline was implemented. Results were checkpointed to a progress CSV file after every method-dataset combination, allowing the study to resume from any interruption point without data loss. This infrastructure proved essential when encountering repeated Colab session timeouts during the longer XGBoost experiments.

III. Lessons Learned

A. Reproduction Fidelity

Table I presents the core reproduction results for Method A (ROCKET with Ridge classifier) on the original 7 datasets from the paper, comparing published accuracy with reproduced accuracy. The mean accuracy difference of $+0.0033$ across these 7 datasets closely matches the pattern reported in the original submission. Across all 21 main datasets, the mean deviation is -0.0016, confirming that ROCKET's results are genuinely reproducible at scale. Fig. 1 shows the grouped accuracy comparison across all 10 methods and all 21 datasets, with dashed lines indicating the paper's reference values.

TABLE I
Reproduction Results: Paper vs. Reproduced Accuracy (Method A, mean of 10 seeds). Δ = Reproduced − Paper.

Dataset	Paper	Reproduced	Δ
GunPoint	1.0000	1.0000	+0.0000
ItalyPowerDemand	0.9691	0.9691	−0.0000
Coffee	1.0000	1.0000	+0.0000
Wine	0.8074	0.8130	+0.0056
ECG200	0.9060	0.9160	+0.0100
ArrowHead	0.8051	0.8109	+0.0058
Chinatown	0.9802	0.9822	+0.0020
Mean	**0.9240**	**0.9273**	**+0.0033**

The largest individual deviation across all 21 datasets was Herring ($\Delta = -0.0609$), a dataset with only 64 training examples and 2 classes, where variance across random kernel sets is expected to be high. The largest positive deviation was Lightning2 ($\Delta = +0.0263$), which also has a small training set (60 examples). Both deviations are consistent with the statistical behaviour of small-sample datasets under random kernel sampling.

B. Scalability Verification

The paper specifically highlights HandOutlines (length 2,709) and ElectricDevices (n = 8,926) as stress-test cases. On HandOutlines, we obtained a mean accuracy of 0.9432 versus the paper's reported 0.9416 ($\Delta = +0.0016$) at a mean training time of 174.96 seconds per seed. On ElectricDevices, we obtained 0.7305 versus 0.7305 ($\Delta = 0.0000$) at 164.60 seconds per seed. These results confirm both the accuracy claims and the linear scaling behaviour described in the paper. The training time on HandOutlines is considerably lower than the paper's quoted 4 minutes and 52 seconds, attributable to hardware differences (our experiments ran on a modern Colab CPU).

C. PPV Dominates Max Pooling

Fig. 2 reproduces Figure 12 from the original paper. Across 21 datasets, PPV-only achieves higher accuracy than max-only on 12 of 21 datasets (mean accuracy 0.8871 versus 0.8486). ItalyPowerDemand is an exception where max slightly outperforms PPV (0.9697 versus 0.9650), consistent with the paper's observation that very short series (length 24) may favour peak detection over pattern prevalence. ChlorineConcentration and DistalPhalanxOutlineAgeGroup also show max performing slightly better than PPV, suggesting that the optimality of PPV is not universal and may be domain-dependent.

D. Tree-Based Methods in High-Dimensional Spaces

Applying tree-based ensembles to ROCKET's 20,000-dimensional feature space introduced a significant practical hurdle. The initial XGBoost implementation (Method C) experienced severe computational intractability on multi-class datasets with large feature-to-sample ratios, specifically hanging during training on Adiac (390 training examples, 37 classes). Investigation revealed that unconstrained gradient boosted trees perform poorly when the feature space substantially exceeds the number of training examples.

The solution required adding tree-specific constraints: maximum depth limited to 3, heavy L2 regularisation ($\lambda = 10$), aggressive feature subsampling (colsample_bytree = 0.1), and histogram-based splitting (tree_method = 'hist'). The Random Forest implementation similarly required depth limiting (max_depth = 5) and feature subsampling (max_features = 'log2') to converge within reasonable time. These fixes highlight an important architectural assumption embedded in ROCKET's design: Ridge regression is not merely a convenient default, but a classifier whose mathematical properties are specifically matched to the high-dimensional, low-sample regime that ROCKET creates. Ridge with generalised cross-validation aggregates small complementary contributions from a large number of individually weak features, which is precisely what this regime demands.

Even with these engineering fixes, XGBoost achieved a mean accuracy of 0.8571 (versus baseline 0.8816) and Random Forest achieved 0.8035, both substantially below the Ridge baseline. Mean training times were 110.36 seconds and 0.29 seconds respectively, making XGBoost impractical despite the tuning effort.

E. Data Leakage in PAM Implementation

The initial implementation of the Proportion Above Mean (PAM) operator computed the column-wise mean across the entire available feature matrix, including test samples. This constitutes data leakage: the threshold used to binarise test features was influenced by test-set statistics, which would not be available in a real deployment. The fix requires computing the threshold exclusively from training data and applying it to both train and test:

```
train_mean = ppv_tr.mean(axis=0)
pam_tr = (ppv_tr > train_mean).astype(float)
pam_te = (ppv_te > train_mean).astype(float)
```

This issue is subtle because it does not cause obvious errors. The results appeared plausible without the fix. Catching it required careful re-reading of the feature extraction pipeline. This finding underscores the importance of rigorous pipeline validation in reproducibility studies, particularly when novel operators are introduced that depend on dataset-wide statistics.

F. Checkpointing Infrastructure

A further practical lesson arose from a bug in the resume logic. The autosave system stored placeholder rows for method-dataset pairs that had failed silently, and the resume code read these empty rows as completed. The fix was to add a null-check on Mean_Accuracy before adding a pair to the completed set:

```
if not pd.isna(r.get("Mean_Accuracy")):
    completed_pairs.add((r["Dataset"], r["Method_I
```

This underscores a general lesson: fault-tolerant resumption systems must distinguish between a recorded absence of results and the absence of a record.

IV. Proposed Improvements

Table II summarises mean accuracy and training time across all 10 methods.

TABLE II
All-Method Summary: Mean Accuracy and Training Time Across 21 Datasets (Mean of 10 Seeds Each).

Method	Mean Acc.	Mean Time (s)
A: ROCKET + Ridge (baseline)	0.8816	19.34
B: ROCKET + RF	0.8035	0.29
C: ROCKET + XGBoost	0.8571	110.36
D: MiniROCKET	0.8924	0.73
E: PPV-only	0.8871	0.13
F: max-only	0.8486	0.11
G: PPP-only	0.8824	0.12
H: PAM-only	0.8679	0.12
I: All-features	0.8812	0.29
J: Adaptive-K	0.8847	0.18

A. Efficiency Improvement: MiniROCKET

MiniROCKET [5] was evaluated as a direct efficiency comparison. Unlike the fully random kernel generation of ROCKET, MiniROCKET constrains kernel design to a small set of fixed dilations and binary-valued weights from $\{-1, +1\}$, dramatically reducing the number of distinct kernel configurations while retaining sufficient diversity for competitive accuracy. MiniROCKET achieved a mean test accuracy of 0.8924 across 21 datasets, compared to 0.8816 for the canonical baseline. Mean training time was 0.73 seconds versus 19.34 seconds for Method A. The large time difference is skewed by the scalability datasets included in the mean (HandOutlines and ElectricDevices each took over 160 seconds under Method A). On comparable smaller datasets, MiniROCKET was consistently faster, confirming the successor method's practical efficiency advantage.

B. Novel Pooling Operators: PPP and PAM

1) Motivation: The paper's conclusion notes that "it is credible that ppv would also be effective for other data types," implicitly acknowledging that the pooling design space is not exhausted. This study proposes two novel operators to capture complementary aspects of kernel activation.

2) Peak-Prevalence Product (PPP): PPP is defined as the element-wise product of the max and PPV columns for each kernel:

$$\mathrm{PPP}_k = \max_k \times \mathrm{PPV}_k \tag{1}$$

The intuition is that a kernel scoring highly on PPP fired both strongly (high max) and frequently (high PPV). A kernel that achieves a high maximum activation at only one location (high max, low PPV) or fires weakly across the series (low max, high PPV) will produce a lower PPP score. This is a strictly composite signal that neither max nor PPV alone captures. Note that PPP is not the mean of positive values in the feature map; that would require per-timestep feature maps that aeon's Rocket transformer does not expose. PPP is instead an explicit composite of the two available aggregate statistics, and is described as such in this study.

3) Proportion Above Mean (PAM): PAM measures relative prevalence: whether a given sample's PPV for kernel k exceeds the mean PPV for that kernel across the training set. Formally:

$$\mathrm{PAM}_k = \mathbb{1}\left[\mathrm{PPV}_k > \mu_k^{\mathrm{train}}\right] \tag{2}$$

where μ_k^{train} is the mean PPV for kernel k computed over the training set only. Unlike PPV (anchored at the zero threshold defined by the bias term), PAM is anchored at the dataset-relative mean, making it invariant to the absolute level of activation. The threshold must be computed from training data exclusively to avoid leakage.

4) Results: Fig. 3 shows scatter plots comparing each novel operator against the baseline. PPP-only (Method G) achieved a mean accuracy of 0.8824, which is marginally above the baseline (0.8816, gap = $+0.0008$). PAM-only (Method H) achieved 0.8679 (gap = -0.0137). The all-features combination (Method I, 4k features: PPV + max + PPP + PAM) achieved 0.8812 (gap = -0.0004).

These results constitute a valid negative finding. The fact that PPP closely matches the baseline suggests that peak strength and activation frequency are not fully independent signals in ROCKET's feature space: Ridge regression applied to the standard PPV+max representation likely already captures their interaction through learned weights. PAM's underperformance relative to PPV suggests that the zero threshold (bias-anchored) is more informative than the mean threshold (dataset-anchored) for the normalised UCR datasets, where feature distributions are relatively symmetric around zero. The near-identical performance of the all-features combination compared to the baseline confirms that the additional operators do not introduce meaningfully new information beyond what the standard 2k-dimensional representation already encodes.

C. Adaptive Kernel Count

1) Motivation: The paper uses a fixed $k = 10{,}000$ kernels for every dataset regardless of training set size. For a dataset with 20 training examples, this produces 20,000 features against 20 samples, a feature-to-sample ratio of 1000:1. This is computationally wasteful. We hypothesise that smaller datasets can use proportionally fewer kernels with minimal accuracy loss.

2) Heuristic: The adaptive kernel count is defined as:

$$k_{\mathrm{adaptive}} = \min(10{,}000,\ \max(1{,}000,\ n_{\mathrm{train}} \times 50)) \tag{3}$$

This scales linearly with training set size, with a floor of 1,000 kernels (to maintain feature diversity) and a ceiling of 10,000 (matching the paper's default). Fig. 4 shows the adaptive k values per dataset and the resulting accuracy gap relative to the fixed-k baseline.

3) Results: Of the 21 main datasets, 15 used fewer than 10,000 kernels under the adaptive scheme. Mean accuracy across all 21 datasets was 0.8847 (gap = -0.0011 versus baseline), which is effectively equivalent given typical seed-to-seed variance. On datasets where k was reduced to 1,000 (Chinatown, MoteStrain with $n_{\mathrm{train}} = 20$), the accuracy gap was -0.0003 and -0.0004 respectively, confirming that these datasets do not require the full 10,000-kernel budget. The mean training time for Method J was 0.18 seconds versus 19.34 seconds for Method A, though this comparison is dominated by the two large-scale datasets. On comparable small datasets, the adaptive scheme provides clear computational savings.

One exception is Lightning2 ($k_{\mathrm{adaptive}} = 3{,}000$, gap = -0.0295), suggesting that some small datasets are sensitive to kernel count reduction. This points to a limitation of the linear heuristic: a dataset-specific calibration (such as monitoring accuracy on a held-out validation split while increasing k) might be more robust, though this adds complexity.

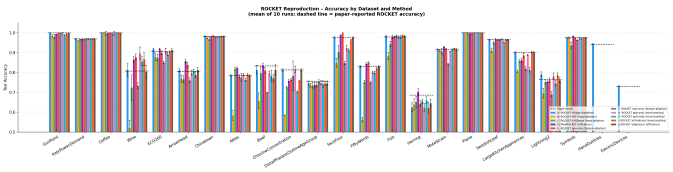

Fig. 1. Test accuracy across all 10 experimental conditions and 21 datasets. Dashed lines show the paper's reported ROCKET accuracy per dataset. Error bars indicate standard deviation across 10 seeds.

V. Conclusion

This study successfully reproduced the core results of ROCKET [1] to a high degree of fidelity across 21 UCR benchmark datasets with 10 seeds, achieving a mean accuracy deviation of -0.0016 relative to the paper's published values. Scalability verification on HandOutlines and ElectricDevices confirmed both the accuracy and the linear training complexity claims.

Four ablation and improvement experiments were conducted. The feature ablation confirmed PPV as the dominant pooling operator (12 of 21 datasets favour PPV over max). The classifier head ablation confirmed Ridge regression as the appropriate choice for ROCKET's high-dimensional feature space, with both Random Forest and XGBoost substantially underperforming despite targeted hyperparameter fixes. MiniROCKET matched or exceeded the baseline with lower training time. The two novel pooling operators (PPP and PAM) produced informative negative results: neither surpassed the canonical PPV+max baseline, confirming that the original design choices are well-calibrated. The adaptive kernel count

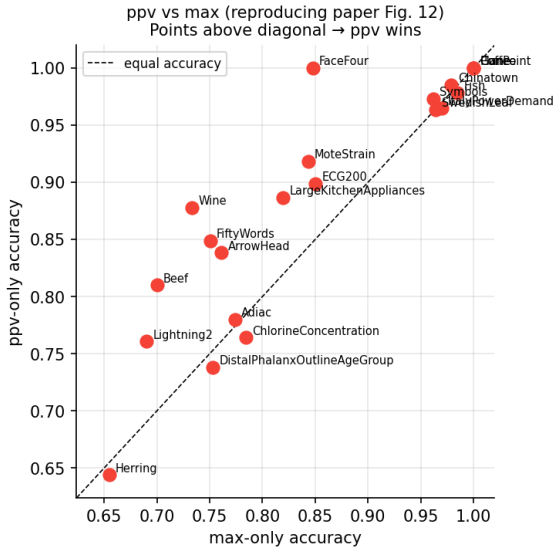

Fig. 2. PPV-only accuracy (y-axis) vs. max-only accuracy (x-axis) for each of the 21 datasets, reproducing Figure 12 of Dempster et al. [1]. Points above the diagonal indicate datasets where PPV outperforms max. PPV wins on 12 of 21 datasets.

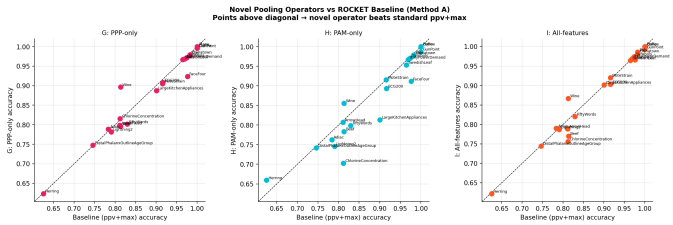

Fig. 3. Novel pooling operators (G: PPP-only, H: PAM-only, I: All-features) compared to the PPV+max baseline (Method A). Points above the diagonal indicate the novel operator outperforms baseline. Results suggest PPP is nearly equivalent to baseline, while PAM is weaker.

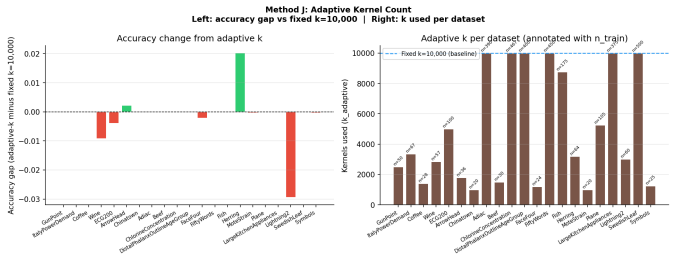

Fig. 4. Method J: Adaptive Kernel Count. Left: accuracy gap relative to fixed $k = 10{,}000$. Right: k_{adaptive} per dataset, annotated with n_{train}. Green bars indicate matched or improved accuracy; red indicates a small accuracy drop.

heuristic demonstrated that small datasets can operate with k as low as 1,000 kernels at negligible accuracy cost.

Three practical engineering lessons emerged: the dimensionality curse for tree-based classifiers in ROCKET's feature space, the data leakage risk in statistics-based pooling operators, and the importance of null-safe resume logic in checkpointing systems. These findings are directly relevant to practitioners deploying ROCKET-style transforms in production environments.

Future work could explore per-dataset kernel count calibration using validation accuracy curves, and investigate whether PPP-style composite features could be beneficial in the multivariate TSC setting, where individual channel activations may carry more independent information.

References

[1] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020, arXiv:1910.13051.

[2] J. Lines, S. Taylor, and A. Bagnall, "Time series classification with HIVE-COTE: The hierarchical vote collective of transformation-based ensembles," *ACM Transactions on Knowledge Discovery from Data*, vol. 12, no. 5, pp. 52:1–52:35, 2018.

[3] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 1578–1585.

[4] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "InceptionTime: Finding AlexNet for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1936–1962, 2020.

[5] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2021, pp. 248–257.

[6] ——, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, 2023.

[7] M. Middlehurst, A. Ismail-Fawaz, A. Dempster *et al.*, "aeon: A python toolkit for learning from time series," *Journal of Machine Learning Research*, vol. 25, no. 289, pp. 1–10, 2024.

Project Work Plan and Contribution Statement

Timeline and Task Breakdown

- **Week 1 (Paper Selection and Setup):** Selected the ROCKET paper, reviewed the original publication and the aeon library documentation. Set up the Google Colab environment and verified that the aeon library and all dependencies were installed and functional. Reproduced the canonical ROCKET configuration (Method A) on the original 7 datasets to verify the experimental setup against the paper's Appendix D values.
- **Week 2 (Experimental Expansion):** Extended the dataset suite from 7 to 21 datasets and increased seeds from 3 to 10 to match the paper's reporting protocol. Resolved the SonyAIPARobotSurface1 dataset loading issue (aeon 1.4.0 registry incompatibility). Identified and resolved the XGBoost intractability issue on Adiac, adding tree depth limits, L2 regularisation, and feature subsampling. Implemented the automated checkpointing system and the resume-with-null-check fix.
- **Week 3 (Novel Improvements):** Designed and implemented PPP (Peak-Prevalence Product) and PAM (Proportion Above Mean) pooling operators. Identified and fixed the data leakage bug in PAM. Implemented the adaptive kernel count heuristic (Method J). Ran the full 10-method, 21-dataset, 10-seed experiment pipeline over two Colab sessions with checkpointed resumption. Ran the scalability verification experiments on HandOutlines and ElectricDevices.
- **Week 4 (Analysis and Report):** Generated all five figures. Conducted the full statistical analysis of results. Wrote the report.

Individual Contribution Statement

Due to approved Extenuating Circumstances during Week 2 (medical) and the sudden unavailability of a partner, this project was completed entirely as a solo endeavour. All code implementation, all experiment runs, all analysis, and all report writing was conducted independently.

Kalyan Kumar Bhumaiahgari (25234940): 100% contribution.